

Test Automation

What to automate, architecture, Playwright, Selenium, Robot Framework and sustainable maintenance.

What is test automation?

Test automation uses code and tools to execute checks repeatedly, compare outcomes and produce evidence. It does not replace critical thinking, exploration or risk analysis; it automates checks that are valuable to repeat.

What is it for?

It reduces repetitive work, accelerates feedback, increases regression frequency, supports CI/CD and frees human time for investigation and judgment-heavy testing. Value comes from reliable feedback, not script count.

When to automate

Good candidates are often stable, repetitive, critical scenarios that run frequently and have objective outcomes. Consider maintenance cost, data, environment, external dependencies and feedback speed.

When not to automate first

Unstable features, disposable prototypes, rarely executed flows, highly subjective validation or scenarios whose automation cost exceeds benefit may be lower priorities. Decide based on risk and return.

Pyramid and layers

A healthy strategy combines checks at different levels. Smaller tests tend to be faster and cheaper; E2E tests cover broad flows but are usually slower and more fragile. The right balance depends on architecture and risk.

Playwright, Selenium, Cypress and Robot Framework

These are options with different characteristics and ecosystems. Playwright provides modern browser and API automation; Selenium has a broad WebDriver ecosystem; Cypress has its own architecture and developer experience; Robot Framework is keyword-driven with extensive libraries.

Automation architecture

Separate data, configuration, locators and reusable rules. Fixtures, Page Objects when appropriate, helpers, environment configuration and naming conventions reduce duplication and make suites easier to evolve.

Flaky tests

A flaky test passes and fails without a meaningful product change. Causes include synchronization, shared data, unstable environments, external dependencies and fragile selectors. Retry may reduce temporary



noise but should not hide the root cause.

Evidence and debugging

Screenshots, videos, traces, logs, console and network data help investigate failures. Automation should produce enough information to answer where, when and why a check failed.

Automation in CI/CD

In pipelines, tests can run on pull requests, merges, builds or schedules. Suites should be organized by speed and risk: fast checks early, heavier suites at appropriate stages, with results and artifacts accessible to the team.